

Desafio de Projeto: Aplicação de Gerenciamento de Tarefas com Docker

Objetivo: Criar uma aplicação de gerenciamento de tarefas que permite aos usuários criar, editar e visualizar tarefas, com autenticação, cache e filas de notificações. A aplicação deve ser implementada usando Vite.js no frontend e Nest.js no backend, com ORM Prisma, cache Redis e filas MQTT. Todo o ambiente deve ser configurado para rodar usando Docker e Docker Compose.

Requisitos Funcionais:

1. **Autenticação e Autorização:**
 - Usuários devem poder se registrar e fazer login.
 - Proteção de rotas para que apenas usuários autenticados possam criar, editar e visualizar tarefas.
2. **Gerenciamento de Tarefas:**
 - Criar, editar, marcar como concluída e deletar tarefas.
 - Listar tarefas com filtros (por status, por data, etc.).
3. **Cache:**
 - Implementar cache usando Redis para armazenar e recuperar a lista de tarefas e o perfil do usuário.
 - O cache deve ser invalidado em caso de alteração nas tarefas ou perfil do usuário.
4. **Filas e Mensageria:**
 - Usar MQTT para gerenciar uma fila de notificações para quando uma nova tarefa for criada. Essas notificações devem ser enviadas para o app do respectivo usuário.
5. **Frontend:**
 - Desenvolver uma interface responsiva utilizando Vite.js e React.
 - A interface deve permitir interação com todas as funcionalidades descritas nos requisitos funcionais.
6. **Mobile:**
 - Desenvolver um aplicativo mobile usando React Native.
 - Implementar telas de login, registro, listagem de tarefas, criação/edição de tarefas e detalhes da tarefa.
 - Integrar com a API do backend para autenticação, gerenciamento de tarefas e recebimento de notificações em tempo real via MQTT.
 - Implementar persistência de login usando AsyncStorage ou similar.
7. **Backend:**
 - Desenvolver o backend com Nest.js.
 - Usar Prisma para gerenciar a persistência de dados no banco de dados (PostgreSQL ou MySQL).
 - Implementar endpoints RESTful para as operações CRUD (Create, Read, Update, Delete) das tarefas e autenticação de usuários.
 - **Testes Unitários e de Integração:**
 - Implementar testes unitários para garantir o correto funcionamento das funcionalidades do backend.
 - Implementar testes de integração para verificar a interação entre componentes e a funcionalidade geral da aplicação.
8. **Docker e Docker Compose:**
 - Configurar Docker para o frontend, backend, Redis, MQTT e banco de dados.
 - Criar um arquivo `docker-compose.yml` para orquestrar todos os serviços.

- Garantir que a aplicação possa ser construída e executada em containers Docker.

9. Documentação e Git:

- Documentar a API com Swagger.
- Criar um README detalhado no repositório do projeto, incluindo instruções para configuração e execução com Docker.
- Utilizar Git para versionamento do código, com commits claros e bem descritos.

Passos Detalhados:

1. Configuração do Backend (Nest.js):

- Configurar um projeto Nest.js com módulos, controladores e serviços.
- Implementar autenticação com JWT (JSON Web Tokens).
- Configurar Prisma para ORM e gerenciar a persistência de dados no banco de dados.
- Integrar Redis para cache e MQTT para filas de mensagens.
- **Implementar Testes:**
 - **Testes Unitários:**
 - Criar testes unitários para serviços, controladores e qualquer lógica de aplicação usando ferramentas como Jest (já integrado com Nest.js).
 - Exemplos de testes unitários incluem verificar se a criação de uma tarefa funciona corretamente, se a autenticação retorna o token correto, e se o cache é atualizado conforme esperado.
 - **Testes de Integração:**
 - Criar testes de integração para garantir que diferentes partes da aplicação interajam corretamente.
 - Exemplos de testes de integração incluem verificar se a comunicação entre o backend e o banco de dados está funcionando, se a autenticação permite o acesso correto às rotas protegidas, e se o sistema de notificações via MQTT está funcionando corretamente.

2. Configuração do Frontend (Vite.js e React/Vue):

- Configurar o Vite.js para desenvolvimento e construção do frontend.
- Implementar um design responsivo usando CSS-in-JS ou pré-processadores como SASS.
- Integrar o frontend com a API do backend para operações de gerenciamento de tarefas e autenticação.
- **Implementar Testes:**
 - **Testes Unitários:**
 - Criar testes unitários para componentes React usando ferramentas como Jest e React Testing Library.
 - Exemplos de testes unitários incluem verificar se os componentes renderizam corretamente, se os formulários de criação de tarefa funcionam e se a lógica de filtro de tarefas está correta.
 - **Testes de Integração:**
 - Criar testes de integração para verificar a interação entre componentes e a integração com a API do backend.
 - Exemplos de testes de integração incluem verificar se a listagem de tarefas é atualizada corretamente ao criar uma

nova tarefa e se a autenticação do frontend interage corretamente com o backend.

3. Docker e Docker Compose:

- **Dockerfile para o Backend:**
 - Criar um Dockerfile que construa e configure o ambiente para o backend Nest.js.
- **Dockerfile para o Frontend:**
 - Criar um Dockerfile que construa e configure o ambiente para o frontend Vite.js.
- **docker-compose.yml:**
 - Configurar serviços para o backend, frontend, Redis, MQTT e banco de dados.
 - Expor portas apropriadas e configurar redes para comunicação entre os serviços.
- **Arquivo .dockerignore:**
 - Adicionar arquivos e pastas que não devem ser incluídos nas imagens Docker (como node_modules, dist, etc.).

4. Testes e Build:

- Testar a aplicação localmente usando Docker Compose para garantir que todos os serviços funcionem conforme esperado.
- Garantir que os testes unitários e de integração sejam executados e passem antes da construção final.
- Documentar o processo de build e execução no README.

5. Documentação e Git:

- Manter um histórico de commits claro e bem estruturado.
- Criar uma documentação de API usando Swagger e fornecer instruções claras no README sobre como configurar e executar a aplicação usando Docker.
- Incluir instruções para a configuração do ambiente de desenvolvimento e produção.

Critérios de Avaliação:

1. Qualidade do Código:

- Clareza e organização do código.
- Uso adequado de padrões e melhores práticas.

2. Funcionalidade:

- Cumprimento dos requisitos funcionais e técnicos.
- Correção e robustez das funcionalidades implementadas.

3. Performance:

- Eficiência do uso de cache e filas.
- Responsividade e performance do frontend.

4. Docker e Docker Compose:

- Correta configuração e funcionamento dos serviços Docker.
- Capacidade de construir e executar a aplicação usando Docker Compose.

5. Documentação:

- Clareza e completude da documentação.
- Facilidade de configuração e execução do projeto a partir das instruções fornecidas.

6. Versionamento com Git:

- Histórico de commits bem estruturado e descritivo.
- Uso de branches para funcionalidades e correções.